Week 8: From Docker to Kubernetes Scheduling

Kubernetes Basics, Distributed Scheduling, and Hands-On Lab

> Week 7 answered: **how do distributed nodes agree on one shared history?** Week 8 asks: **who decides where containers run — and how?**

Today's Road Map

Part	Topic	Key Question
1	Kubernetes Basics	What is K8s, and why do we need it beyond Docker?
2	K8s Architecture	What are the control-plane components and how do they interact?
3	Distributed System Scheduling	How does kube-scheduler decide where Pods run?
4	Scheduling Deep Dive	Filtering, scoring, fairness, and fragmentation
5	Lab 8 — K8s Scheduling	Hands-on: observe scheduling decisions in a live cluster

The OS question: a single kernel schedules threads on one CPU. **What replaces that when you have hundreds of machines and thousands of containers?**

Part 1

Kubernetes Basics — Why Docker Is Not Enough

Recap: Docker Solves a Strong but Narrow Problem

Docker gives us:

- packaging and image distribution (images, OverlayFS layers)
- isolation via namespaces and cgroups
- a path from `docker run` to a running process on **one machine**

```
$ docker run -it --cpus=1 --memory=512m nginx

docker CLI → dockerd → containerd → runc
  → clone(CLONE_NEWNS | CLONE_NEWPID | CLONE_NEWNET | ...)
```

```
  → write cpu.max, memory.max to cgroup files
  → execve("/usr/sbin/nginx")
```

Docker is not a kernel feature. It is a user-space toolchain that calls the kernel APIs we studied in earlier weeks.

What Docker Cannot Do

Docker is a **single-node tool**. It does not answer:

```
What Docker CAN do:                    What Docker CANNOT do:
──────────────────────                 ────────────────────────
✓ Package app + deps into image        ✗ Decide which machine to run it on
✓ Isolate with namespaces + cgroups    ✗ Restart it on another machine if
one dies
✓ Limit CPU and memory                 ✗ Load-balance traffic across copies
✓ Fast startup via OverlayFS layers    ✗ Scale up/down based on load
✓ Reproducible builds                  ✗ Coordinate 100 services across 50
machines
```

> If you have 1 machine and 5 containers, Docker is enough. If you have 100 machines and 5000 containers, you need **orchestration**.

The Problems of Scale

Scenario: you run a web service with 10 replicas across 5 machines

```
Problem 1 — SCHEDULING:
  "Which machine should replica #7 run on?"
  SSH into each machine and `docker run`? Doesn't scale.

Problem 2 — SELF-HEALING:
  Machine #3 dies at 3 AM. Replicas on it are gone.
  Who detects this? Who restarts them on surviving machines?

Problem 3 — SERVICE DISCOVERY:
  Your web service needs to talk to the database.
  The database moved after a restart. How does the service find it?

Problem 4 — ROLLING UPDATES:
  Deploy v2 without downtime. Start new replicas, verify health,
  drain old replicas. By hand across 5 machines = outage recipe.
```

```
Problem 5 — RESOURCE EFFICIENCY:
  Machine #1 at 90% CPU, machine #5 at 10%. Nobody is balancing.
```

These are the problems Kubernetes solves.

How Kubernetes Solves Each Problem

Problem	Docker World	Kubernetes Solution
Scheduling	SSH + `docker run` by hand	**kube-scheduler** automatically picks the best node based on resources, affinity, and constraints
Self-Healing	Nobody restarts at 3 AM	**controller manager** detects node/pod failure, creates replacement Pods on surviving nodes within seconds
Service Discovery	Hardcoded IP / manual DNS	**Service** + **kube-dns**: every Service gets a stable virtual IP and DNS name (`my-db.default.svc.cluster.local`)
Rolling Updates	Manual stop-start across machines	**Deployment** `strategy: RollingUpdate` — adds new replicas, health-checks, drains old ones, zero-downtime by default
Resource Efficiency	No cross-node visibility	Scheduler scores nodes by utilization (`LeastAllocated` / `MostAllocated`), bin-packs or spreads automatically

> Each of these is a **reconciliation loop** — a controller that watches the gap between desired and actual state and takes action continuously.

What Is Kubernetes?

Kubernetes (K8s) is an **open-source container orchestration platform** originally designed by Google, now maintained by the Cloud Native Computing Foundation (CNCF).

In one sentence:

> K8s is a **distributed control plane** that decides **where** containers run, **ensures** they keep running, and **manages** their resources using the same kernel primitives (namespaces, cgroups) underneath.

Origin story:

Google ran an internal system called **Borg** that scheduled millions of containers across their data centers since 2003. Kubernetes is the open-source successor, sharing Borg's core ideas: declarative configuration, reconciliation loops, and scheduler-based placement.

The Key Idea: Declarative, Not Imperative

```
Imperative (Docker):                      Declarative (Kubernetes):
────────────────────                      ─────────────────────────
"Run this container on this machine"       "I want 3 replicas of this
service"
"Stop that container"                      "I want zero downtime during
updates"
"Restart it if it crashes"                 "I want each replica to have 500m
CPU"

You tell Docker WHAT TO DO.               You tell K8s WHAT YOU WANT.
Docker does it once.                      K8s continuously works to make it
true.
```

The reconciliation loop:

```
while True:
    desired_state = read_from_api_server()   # what user declared in YAML
    current_state = observe_cluster()         # actual state
    if current_state != desired_state:
        take_action()   # create / delete / restart containers
    sleep(interval)
```

This is the same pattern as a **feedback control system**. If a node dies, desired state hasn't changed — K8s reschedules the lost containers automatically.

Kubernetes as a Distributed OS

Classic OS concept	Kubernetes analogue	Kernel mechanism
Process	Pod (group of containers)	`task_struct` + `nsproxy`
CPU scheduler (`pick_next_task`)	kube-scheduler	N/A (user-space decision)
`nice` / priority	QoS classes + PriorityClass	`cpu.weight` in cgroup
`ulimit` / `cgroup cpu.max`	Pod `limits.cpu`	kubelet writes `cpu.max`
Page fault → `mem_cgroup_charge()`	Pod `limits.memory`	kubelet writes `memory.max`
OOM killer	cgroup OOM + kubelet eviction	kernel kills process; kubelet kills pod
`/proc` + `ps`	`kubectl top` + metrics-server	reads `cpu.stat`, `memory.current`

> A kernel manages one machine directly. Kubernetes manages a fleet **indirectly** through persistent objects and control loops.

Part 2

K8s Architecture — The Control Plane

The Three-Layer Architecture

paxos-5

The control plane makes **decisions**. The worker nodes **execute** them. Every component communicates through the **API server** — there is no direct communication between scheduler and kubelet.

Who Does What in the Control Plane

Component	Main job	OS analogy
API server	Front door for all state changes; validates, authenticates, persists	System call interface
etcd	Durable replicated key-value store (Raft-based, from Week 7)	Kernel object table
controller manager	Turn desired state into concrete objects (e.g., Deployment → Pods)	Kernel threads that maintain invariants
kube-scheduler	Choose a feasible and preferred node for each Pod	CPU scheduler
kubelet	Realize assigned Pods on one node; write cgroup files, call containerd	Per-CPU dispatch + enforcement
kube-proxy	Maintain network rules for Service routing	iptables / ipvs configuration

The scheduler is important, but it is only **one stage** in a larger control loop.

Core Abstractions: Pod, Deployment, Service

Pod — the smallest schedulable unit

```
apiVersion: v1            # which API group — core/v1 for Pods
kind: Pod                 # object type — tells API server what to create
metadata:
```

```
  name: my-app            # unique name within this namespace
  labels:                 # key-value tags — used by Services and
Deployments to FIND this Pod
    app: my-app
spec:
  containers:
  - name: web             # container name (for logs: kubectl logs my-app -
c web)
    image: nginx:1.25     # OCI image — pulled by kubelet → containerd
    resources:
      requests:           # ── SCHEDULER looks at these (accounting) ──
        cpu: "500m"       #   500 millicores = 0.5 CPU → sets cpu.weight in
cgroup
        memory: "256Mi"   #   256 MiB → sets memory.min (guaranteed
minimum)
      limits:             # ── KERNEL enforces these (cgroup hard caps) ──
        cpu: "1000m"      #   1 full CPU → writes cpu.max = "100000 100000"
        memory: "512Mi"   #   512 MiB → writes memory.max; OOM kill if
exceeded
    ports:
    - containerPort: 80   # informational — tells readers which port the
app listens on
```

A Pod = one or more containers that share the **same network namespace** and can share **volumes**.

Core Abstractions (continued)

Deployment — "I want N replicas, always running"

```
apiVersion: apps/v1         # apps group — higher-level controllers live
here
kind: Deployment            # manages ReplicaSets which manage Pods
metadata:
  name: my-app
spec:
  replicas: 3               # desired count — controller creates/deletes
Pods to match
  selector:                  # ┐
    matchLabels:             # │ HOW the Deployment finds "its" Pods:
      app: my-app            # │ "only manage Pods whose labels include
app=my-app"
  template:                  # ┘ ← Pods are created FROM this template
    metadata:
      labels:
        app: my-app          # ← MUST match selector above, otherwise
controller can't find them
    spec:
      containers:
```

```
        - name: web
          image: nginx:1.25
```

The Deployment controller watches the cluster and ensures exactly 3 Pods exist. If one dies, it creates a replacement. If you change the image tag, it performs a **rolling update**.

A Day in the Life of `kubectl apply`

```
$ kubectl apply -f deployment.yaml

     ┌──────────┐
     │  kubectl  │  "send YAML to API server"
     └──────────┘
          │
          │ HTTPS
          ▼
     ┌──────────┐
     │ API server│  "validate, authenticate, store in etcd"
     └──────────┘
          │
          │ watch event: "new Deployment created"
          ▼
     ┌──────────────────┐
     │ Deployment        │  "desired: 3 replicas. actual: 0. create 3 Pods"
     │ controller        │
     └──────────────────┘
          │
          │ creates 3 Pod objects in API server
          ▼
     ┌──────────┐
     │ Scheduler │  "Pod X has no nodeName. Node A has capacity. Bind."
     └──────────┘
          │
          │ writes pod.spec.nodeName = "node-a"
          ▼
     ┌──────────┐
     │  kubelet  │  "new Pod assigned to me. containerd → runc → cgroups"
     │ (node-a)  │  "container running ✓"
     └──────────┘
```

Week 7: etcd and Consensus

All of the state in that pipeline flows through **etcd** — the Raft-based replicated store from Week 7.

```
scheduler  → API server → etcd     (persist binding decision)
controller → API server → etcd     (persist new Pod objects)
kubelet    → API server → etcd     (persist status updates)
kubectl    → API server → etcd     (persist user intent)
```

Week 7 mechanism	Week 8 consequence
Quorum intersection	No split-brain scheduling decisions
Leader election	Writes resume after control-plane failure
Log replication	Pod objects and bindings survive crashes
Linearizable ops	Scheduler acts on committed, consistent state

> Without Week 7's consensus machinery, the scheduler would have no trustworthy state to read from or write to.

Part 3

Distributed System Scheduling

From CPU Scheduling to Cluster Scheduling

Dimension	Linux CPU scheduler (CFS)	K8s cluster scheduler
Scope	One machine, many cores	Many machines, many resources
Execution unit	Thread	Pod
Main resource	CPU time	CPU, memory, topology, GPUs
State view	Local and immediate (kernel memory)	API-driven and slightly stale (watch events)
Decision	`pick_next_task_fair()`	Filter → Score → Bind
Frequency	Every few milliseconds	Per Pod creation (ms to seconds)
Preemption	Higher-priority task preempts	Higher PriorityClass preempts
Failure model	Task exits	Node may disappear entirely
Bad decision cost	Latency / fairness issue	Fragmentation, availability loss

Key difference: CFS runs in **kernel space** with microsecond decisions. kube-scheduler runs in **user space** and can afford more complex algorithms.

Why Cluster Scheduling Is Harder

A cluster scheduler must reason about all of these at once:

- **Hard feasibility** — can this Pod fit on any node at all?

- **Soft preference** — among feasible nodes, which is better?
- **Multi-resource reasoning** — CPU and memory must be satisfied together
- **Fragmentation** — enough total resources exist, but not in the right shape on any single node
- **Stale views** — the scheduler works from watched state, not direct hardware truth
- **Node failures** — a placement decision can become invalid if the target node crashes

This is why Kubernetes scheduling is not just "bigger CFS." It mixes scheduling, admission control, bin packing, and distributed systems reasoning.

Example: Why Cluster Scheduling Is Hard

Imagine scheduling a **ML training Pod** that needs 4 CPU + 8 GiB + GPU:

```
Cluster state (3 worker nodes):
  Node A:  8 CPU, 16 Gi memory, 1 GPU   | used: 5 CPU, 10 Gi, 0 GPU
  Node B:  8 CPU, 16 Gi memory, 0 GPU   | used: 2 CPU,  4 Gi, 0 GPU
  Node C:  8 CPU, 16 Gi memory, 1 GPU   | used: 7 CPU, 14 Gi, 1 GPU
           ─── taints: Node A has "team=ml:NoSchedule" ───

New Pod: requests 4 CPU, 8 Gi memory, 1 GPU
         tolerates taint "team=ml"
         preferred affinity: zone=us-east-1a (Node A and B are in us-east-
1a)

Scheduling decision:
  Node A: CPU 8-5=3 < 4 → ✗ FILTERED (insufficient CPU, even though GPU is
free!)
  Node B: has 6 CPU, 12 Gi free, but 0 GPU → ✗ FILTERED (no GPU)
  Node C: CPU 8-7=1 < 4 → ✗ FILTERED (insufficient CPU)

Result: Pod stays Pending!
  Total cluster free: 8 CPU, 18 Gi, 1 GPU — ENOUGH in aggregate
  But no single node has the right COMBINATION → this is FRAGMENTATION
```

> **Fragmentation** is cluster scheduling's biggest hidden enemy — total resources look sufficient, but they're scattered across nodes in unusable shapes.

What kube-scheduler Actually Decides

The scheduler is **not** a CPU dispatcher. It does not context-switch threads.

Instead, it repeatedly does this:

```
while True:
    1. Watch for Pods with no nodeName (unscheduled)
```

```
    2. For each unscheduled Pod:
       a. FILTER: which nodes can legally run this Pod?
       b. SCORE:  among legal nodes, which is best?
       c. BIND:   write the chosen nodeName back to the API server
```

The scheduler operates on **objects in cluster state**, not directly on processes. The binding is a **state update** persisted through etcd — exactly where Week 7 consensus comes in.

The Scheduling Pipeline: Filter → Score → Bind

```
Pod created (no nodeName) → scheduler watches API server
    │
    ▼
┌──────────────────────────────────────────────────────────┐
│  1. FILTERING (predicates)                               │
│     "Which nodes CAN run this Pod?"                      │
│     - NodeResourcesFit: allocatable - Σrequests ≥ pod    │
│     - NodeAffinity: does node match affinity rules?      │
│     - TaintToleration: does pod tolerate node taints?    │
│     - PodFitsHostPorts: are requested ports available?   │
│     Result: feasible node set (may be empty → Pending)   │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│  2. SCORING (priorities)                                 │
│     "Which feasible node is BEST?"                       │
│     Each plugin scores 0–100. Weighted sum = winner.     │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│  3. BINDING                                              │
│     Write pod.spec.nodeName = winner                     │
│     kubelet on that node picks it up → containers start  │
└──────────────────────────────────────────────────────────┘
```

Part 4

Scheduling Deep Dive — Filtering, Scoring, and Fairness

Filtering = Hard Constraints

Typical reasons a node is **filtered out**:

- **Insufficient CPU or memory**: `node.allocatable - Σrequests < pod.requests`
- **Node selector mismatch**: Pod asks for `disktype: ssd`, node doesn't have that label
- **Node affinity mismatch**: Pod has `requiredDuringSchedulingIgnoredDuringExecution` rules
- **Untolerated taint**: Node has `key=value:NoSchedule`, Pod lacks matching toleration
- **Host port conflict**: Requested port already in use on the node
- **Topology or volume restriction**: PV is in a different zone

These are easy to diagnose because Kubernetes exposes explicit events: `FailedScheduling`, `Insufficient cpu`, `Insufficient memory`, `didn't match Pod's node affinity/selector`.

Taints and Tolerations — A Key Filtering Mechanism

Taints are applied to **nodes** to repel Pods. Tolerations are declared in **Pods** to accept taints.

```
# Taint a node (repels all Pods without a matching toleration)
kubectl taint nodes worker-1 gpu=true:NoSchedule
```

```
# Pod that tolerates the taint
spec:
  tolerations:
  - key: "gpu"
    operator: "Equal"
    value: "true"
    effect: "NoSchedule"
  containers:
  - name: ml-training
    image: tensorflow/tensorflow:latest-gpu
```

Taint effect	Behavior
`NoSchedule`	New Pods without toleration are **not scheduled** here
`PreferNoSchedule`	Scheduler **tries to avoid** but may still place here
`NoExecute`	Existing Pods without toleration are **evicted**

Node Affinity — Attracting Pods to Nodes

Node affinity is the **opposite** of taints: it attracts Pods to specific nodes.

```
spec:
  affinity:
    nodeAffinity:
```

```yaml
        requiredDuringSchedulingIgnoredDuringExecution:
          nodeSelectorTerms:
          - matchExpressions:
            - key: topology.kubernetes.io/zone
              operator: In
              values:
              - us-east-1a
              - us-east-1b
        preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 80
          preference:
            matchExpressions:
            - key: disktype
              operator: In
              values:
              - ssd
```

- **required** = hard constraint (filtering). Pod stays Pending if no match.
- **preferred** = soft constraint (scoring). Influences ranking but doesn't block.

Scoring = Soft Preferences

After filtering, several nodes may still be legal. The scheduler ranks them.

Scoring Plugin	What it scores	Strategy
`LeastAllocated`	Prefer nodes with most free resources	**Spread** workloads
`MostAllocated`	Prefer nodes with least free resources	**Pack** for efficiency
`NodeResourcesBalancedAllocation`	Prefer nodes where CPU% and mem% are similar	Balance resource dimensions
`InterPodAffinity`	Co-locate or spread based on labels	Application topology
`TaintToleration`	Prefer nodes with fewer untolerated taints	Minimize constraint friction

Spread vs Pack — the fundamental tradeoff

```
Spread (LeastAllocated):                Pack (MostAllocated):
┌──────┐  ┌──────┐  ┌──────┐           ┌──────┐  ┌──────┐  ┌──────┐
│ ■■   │  │ ■■   │  │ ■■   │           │■■■■■■│  │■■■■■■│  │      │
│ ■■   │  │ ■■   │  │ ■■   │           │■■■■■■│  │■■    │  │      │
└──────┘  └──────┘  └──────┘           └──────┘  └──────┘  └──────┘
 Node A    Node B    Node C             Node A    Node B    Node C
```

```
✓ headroom for bursts                    ✓ fewer nodes needed (cost savings)
✗ more nodes running (cost)              ✗ no headroom (cascading failure
risk)
```

Example: InterPodAffinity

Unlike NodeAffinity (which looks at **node labels**), InterPodAffinity looks at **what Pods are already running on a node**.

```
spec:
  affinity:
    podAffinity:                    # Attract: "schedule me NEAR these
Pods"
      requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchExpressions:
          - key: app
            operator: In
            values:
            - redis                 # "find nodes that already run
app=redis"
        topologyKey: kubernetes.io/hostname   # granularity = same NODE
    podAntiAffinity:                # Repel: "schedule me AWAY from these
Pods"
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchExpressions:
            - key: app
              operator: In
              values:
              - web                 # "avoid nodes that already run
app=web"
          topologyKey: topology.kubernetes.io/zone   # granularity = same
ZONE
```

`topologyKey` controls the meaning of "near" / "away": `hostname` = same node, `zone` = same availability zone, or custom labels like `rack`

InterPodAffinity — Use Cases

Two typical scenarios:

```
Scenario 1 — Co-locate for low latency:          Scenario 2 — Spread for
high availability:
  Web frontend Pod + Redis cache Pod                3 replicas of the same
web service
  → podAffinity: app=redis                          → podAntiAffinity:
app=web
  → topologyKey: hostname (same node)               → topologyKey: zone
(different AZs)

  ┌── Node A ──┐  ┌── Node B ──┐                ┌ Zone A ┐  ┌ Zone B ┐
┌ Zone C ┐
  │ [web] [redis] │  │            │                │ [web₁] │  │ [web₂] │
│ [web₃] │
  └────────────┘  └────────────┘                └────────┘  └────────┘
└────────┘
  ✓ minimal network latency                        ✓ zone failure only
loses 1 replica
```

NodeAffinity vs InterPodAffinity:

	NodeAffinity	**InterPodAffinity**
Looks at	Node's own labels (`disktype=ssd`)	Labels of Pods running on that node
Question	"What kind of node do I want?"	"Who do I want to be near / far from?"
Use case	Hardware requirements	Application topology (latency, HA)

Scheduling Example: Tracing the Decision

```
Cluster state:
  Node A: 8 CPU allocatable, 4 CPU already requested
  Node B: 8 CPU allocatable, 6 CPU already requested
  Node C: 8 CPU allocatable, 8 CPU already requested (full)

New Pod: requests.cpu = "2000m" (2 CPUs)

Step 1 — Filtering:
  Node A: 8 - 4 = 4 available ≥ 2 → PASS
  Node B: 8 - 6 = 2 available ≥ 2 → PASS
  Node C: 8 - 8 = 0 available < 2 → FAIL (filtered out)

Step 2 — Scoring (LeastAllocated):
  Node A: (8 - 4 - 2) / 8 = 25% remaining → score 25
  Node B: (8 - 6 - 2) / 8 =  0% remaining → score  0

Step 3 — Binding:
```

```
    Winner: Node A → write pod.spec.nodeName = "node-a"
    kubelet on Node A picks up the Pod → creates containers
```

If no node passes filtering: Pod stays `Pending`. `kubectl describe pod` shows: `0/3 nodes are available: insufficient cpu`.

What "Pending" Really Means

When a Pod is `Pending`:

```
The scheduler computed:
  For every node N:
    N.allocatable.cpu - Σ(pod requests on N).cpu < new_pod.requests.cpu

This is an ACCOUNTING problem, not an enforcement problem.
```

The distinction matters:

- `requests` are **promises** (accounting) — used by the **scheduler**
- `limits` are **enforcement** (cgroups) — used by the **kernel**
- A node can have 0 allocatable CPU (all requests claimed) but still have idle CPU (pods aren't using their full requests)

This is why **overcommit** exists: total requests can exceed node capacity. It works because not all pods use their full requests simultaneously — just like memory overcommit in a traditional OS.

> Pending is not boring. It is the visible symptom of a **cluster scheduling decision**.

Fairness, Packing, and Fragmentation

Goal	Why we want it	What can go wrong
Fairness	Small users and short jobs should not starve	Large jobs dominate waiting time
Packing	Better consolidation improves utilization	Interference and hot spots
Low fragmentation	Preserve room for future big jobs	Greedy placement scatters free capacity

A real scheduler is not optimizing one metric. It navigates these tensions under partial information.

Backfilling: A Classic HPC Response to Head-of-Line Blocking

If the first job in a queue cannot start yet, a strict FIFO scheduler may leave resources idle.

Backfilling allows the scheduler to run a smaller job earlier **as long as it does not delay the reserved start time of the blocked job**.

- big job at queue head keeps its reservation
- short job uses the temporary gap
- utilization improves without fully abandoning queue order

Why Backfill Matters for This Week's Lab

Our Lab 8 simulation compares **FIFO** and **Backfill** because they expose an important cluster-scheduling tension:

- **FIFO** is simple and easy to reason about
- **Backfill** reduces idle gaps and head-of-line blocking
- but backfill depends on some estimate of how long jobs will run

This is **not** how Kubernetes schedules Pods directly, but it is a useful bridge from:

- single-resource CPU scheduling
- to multi-node batch scheduling
- to Kubernetes placement under fairness and fragmentation constraints

---

Priority and Preemption

K8s supports **PriorityClasses** — a cluster-level priority that affects scheduling order and preemption.

```yaml
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata:
  name: high-priority
value: 1000000
globalDefault: false
description: "For critical production workloads"
---
apiVersion: v1
kind: Pod
metadata:
  name: critical-pod
spec:
```

```
  priorityClassName: high-priority
  containers:
  - name: web
    image: nginx
```

When a high-priority Pod cannot be scheduled:

1. Scheduler identifies lower-priority Pods whose removal would free resources
2. Those Pods are **preempted** (gracefully terminated)
3. High-priority Pod is scheduled in their place

> Analogous to real-time priority preemption in Linux CFS.

The Scheduling Framework — Plugin

Since K8s v1.19, the scheduler uses a **plugin-based Scheduling Framework**. Every scheduling decision passes through a series of **extension points** — each point runs one or more plugins.

```
                        SCHEDULING CYCLE (synchronous, per-Pod)

┌──────────────────────────────────────────────────────────────────────────┐
  │ Sort → PreFilter → Filter → PostFilter → PreScore → Score
│
  │                                          → NormalizeScore → Reserve
│

└──────────────────────────────────────────┬───────────────────────────────┘
                                           │ winner chosen
                        BINDING CYCLE      │ (asynchronous, can overlap next Pod)

┌──────────────────────────────────────────▼───────────────────────────────┐
  │ Permit → PreBind → Bind → PostBind
│

└──────────────────────────────────────────────────────────────────────────┘
```

- **Scheduling cycle** is serial: only one Pod is scored at a time (avoids race conditions on node capacity)
- **Binding cycle** is async: writing `nodeName` to etcd can overlap with the next Pod's scheduling cycle
- Will this lead to contention?

Extension Points Explained

Extension Point	When it runs	Example Plugin
Sort	Order Pods in the scheduling queue	`PrioritySort` — higher priority first
PreFilter	Pre-check / compute data for all nodes at once	`InterPodAffinity` — build topology map
Filter	Per-node: can this node run the Pod?	`NodeResourcesFit`, `TaintToleration`
PostFilter	No node passed Filter — attempt rescue	`DefaultPreemption` — try evicting lower-priority Pods
PreScore	Prepare shared state for scoring	`InterPodAffinity` — compute spread map
Score	Per-node: rate 0–100	`LeastAllocated`, `NodeAffinityPriority`
NormalizeScore	Scale scores to [0, 100]	Built-in normalization
Reserve	Optimistically reserve resources on chosen node	`VolumeBinding`
Permit	Approve, deny, or delay (wait-on-condition)	Gang scheduling plugins
PreBind / Bind / PostBind	Actually write the binding to API server	`DefaultBinder`

> You can write **custom plugins** in Go and register them at any extension point — this is how companies like ByteDance, Alibaba, and Apple customize K8s scheduling.

The Scheduling Queue: Three Buckets

Pods waiting to be scheduled live in one of three queues:

```
   new Pod ──►  ┌──────────────┐  ◄── Pods ready to be scheduled (sorted
by priority)
                │   ActiveQ    │
                └──────┬───────┘
                       │ scheduling fails
                       ▼
                ┌──────────────┐
                │  BackoffQ    │  ◄── retry after exponential backoff
                │  (1s→2s→4s   │      (avoids hot-looping on unschedulable
Pods)
                │   ...→60s)   │
                └──────┬───────┘
                       │ still fails after retries
```

```
                        ▼
                  ┌──────────────────┐
                  │ Unschedulable    │  ◀── parked here until cluster state
changes
                  │ Pods             │      (new node added, Pod deleted,
etc.)
                  └──────────────────┘
                        │ cluster event triggers re-evaluation
                        ▼
                    back to ActiveQ
```

Why this matters: without backoff, a single unschedulable Pod could consume 100% of scheduler CPU. The three-bucket design ensures the scheduler stays responsive for Pods that *can* be scheduled.

Scheduling Cycle vs Binding Cycle

```
Timeline ──────────────────────────────────────────────────────────▶

  Pod A: ├── scheduling cycle ──┤──────── binding cycle ────────┤
                                ↑
  Pod B:                        ├── scheduling cycle ──┤──── binding cycle
────┤

  Pod C:                                                  ├── scheduling
cycle ──┤
```

Scheduling cycle (synchronous, serialized):

- Runs Filter + Score for one Pod at a time
- Must complete before next Pod can start scheduling
- Why serial? Because node capacity accounting must be consistent

Binding cycle (asynchronous, parallelized):

- Writes `pod.spec.nodeName` to API server → etcd
- Can run in background while next Pod is being scheduled
- If binding fails (node died, conflict), Pod goes back to ActiveQ

> This is analogous to the **commit phase** in a database — you serialize the decision but pipeline the persistence.

Part 5

Lab 8 — Kubernetes Scheduling Hands-On

Lab 8 Overview

In this lab you will:

1. Implement **FIFO** and **Backfill** in a small scheduler simulation
2. Compare makespan, completion time, wait time, and utilization
3. Create a local Kubernetes cluster using `kind`
4. Watch scheduling events as Pods move from creation to binding
5. Make **scoring** visible with a soft preference
6. Trigger `Pending` and diagnose hard **filtering** failures
7. Free resources and observe recovery

Prerequisites

```
# Check required tools
docker version
kind version
kubectl version --client
python3 --version
```

If any tool is missing, follow the installation steps in `lab8_scheduling_instructions.md`.

Simulation Path: FIFO vs. Backfill

Part A/B is deliberately small and controllable.

We compare:

- **Makespan** — how long the whole workload takes
- **Avg completion time** — how quickly jobs finish on average
- **Avg wait time** — how long jobs sit in queue
- **Utilization** — how much cluster capacity is actually used

The point is not one perfect metric. The point is to see how one policy changes multiple outcomes.

Why We Use a Simple Workload

The simulation is intentionally simpler than real Kubernetes:

- all jobs use a **single machine**
- **FIFO** gives a clear baseline
- **Backfill** shows how smaller jobs can use temporary gaps
- this makes head-of-line blocking visible without too much code complexity

---

Part C Setup: Small kind Cluster

Use a plain **1 control-plane + 2 worker** cluster.

```
# Save as kind-scheduling-lab.yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
- role: worker
- role: worker
```

```
kind create cluster --name sched-lab --config kind-scheduling-lab.yaml
kubectl cluster-info
kubectl get nodes -o wide
kubectl create namespace lab8-sched
```

Observe Scheduling Events First

Open one terminal for events:

```
kubectl get events -n lab8-sched --watch --field-selector reason=Scheduled
```

Optional: in a second terminal, keep a live Pod view open:

```
kubectl get pods -n lab8-sched -o wide -w
```

Then create a tiny Pod:

```
kubectl run small-job -n lab8-sched --image=busybox --restart=Never \
  --overrides='{"spec":{"containers":[{"name":"small-
job","image":"busybox","command":["sleep","300"],"resources":{"requests":
{"cpu":"100m","memory":"64Mi"}}}]}}'

kubectl get pod -n lab8-sched small-job -o wide
```

Why Filtering Is Easier to See Than Scoring

In a tiny cluster, many nodes may look almost identical.

That means:

- **filtering** is easy to observe from `FailedScheduling` events
- **scoring** is often hidden unless we create a soft preference

So the lab deliberately includes one example for each:

- one soft preference for **score**
- one resource failure for **filter**

Make Scoring Visible with a Soft Preference

```
kubectl label node sched-lab-worker disk=ssd --overwrite
kubectl label node sched-lab-worker2 disk=hdd --overwrite
```

```
kubectl apply -n lab8-sched -f - <<'EOF'
apiVersion: v1
kind: Pod
metadata:
  name: prefer-ssd
spec:
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        preference:
          matchExpressions:
          - key: disk
            operator: In
            values: ["ssd"]
  containers:
  - name: bb
    image: busybox
    command: ["sleep", "300"]
    resources:
      requests:
        cpu: "100m"
        memory: "64Mi"
EOF

kubectl get pod -n lab8-sched prefer-ssd -o wide
```

If both workers are feasible, `prefer-ssd` should usually land on `sched-lab-worker`.

Trigger Pending and Diagnose Filtering

Fill the workers with medium Pods, then try a larger Pod:

```
for i in $(seq 1 6); do
  kubectl run filler-$i -n lab8-sched --image=busybox --restart=Never \
    --overrides="{\"spec\":{\"containers\":
[{\"name\":\"filler-$i\",\"image\":\"busybox\",\"command\":
[\"sleep\",\"600\"],\"resources\":{\"requests\":
{\"cpu\":\"500m\",\"memory\":\"256Mi\"}}}]}}"
done

kubectl run big-job -n lab8-sched --image=busybox --restart=Never \
  --overrides='{"spec":{"containers":[{"name":"big-
job","image":"busybox","command":["sleep","300"],"resources":{"requests":
{"cpu":"2","memory":"1Gi"}}}]}}'
```

Then inspect:

```
kubectl get pod -n lab8-sched big-job
kubectl describe pod -n lab8-sched big-job | grep -A 10 Events
kubectl get events -n lab8-sched --field-selector reason=FailedScheduling
```

What to Look For in the Pending Case

Record:

- Which exact `FailedScheduling` message appeared?
- Which **filter** rejected the Pod?
- Is the cluster actually full, or just **fragmented**?

This is the concrete bridge back to lecture:

- `Insufficient cpu` / `Insufficient memory` ↔ `NodeResourcesFit`
- no feasible node set ↔ Pod stays `Pending`

Recovery Matters Too

Now delete some filler Pods and watch the blocked Pod move forward:

```
kubectl delete pod -n lab8-sched filler-1 filler-2 filler-3
```

```
kubectl get pod -n lab8-sched big-job -w
```

This is important pedagogically:

- `Pending` is not the end of the story
- the scheduler keeps reevaluating cluster state
- once resources become feasible, the Pod can be bound

Optional: Scheduler Logs

```
kubectl logs -n kube-system \
  $(kubectl get pods -n kube-system -l component=kube-scheduler \
    -o jsonpath='{.items[0].metadata.name}') | tail -30

# Fallback inside kind
docker exec sched-lab-control-plane sh -lc 'ls
/var/log/containers/*scheduler* >/dev/null 2>&1 && tail -30
/var/log/containers/*scheduler*'
```

Use logs as supporting evidence, not as the main observability mechanism.

Lab 8 Deliverables

For your lab report, include:

1. Your `schedule_fifo()` and `schedule_backfill()` implementations
2. The comparison table for makespan / completion / wait / utilization
3. Which jobs benefited most from Backfill, and what trade-off you observed
4. The `FailedScheduling` message for `big-job`
5. A note on where `prefer-ssd` landed and what that says about scoring
6. A short Week 7 → Week 8 explanation: when did the scheduling decision become durable shared state?

Cleanup

```
kind delete cluster --name sched-lab
```

Summary

1. Docker solves **single-node** container isolation. Kubernetes adds **multi-node orchestration**: scheduling, self-healing, and resource management.
2. K8s is a **distributed control plane** built on the same kernel primitives (namespaces, cgroups) — automated through kubelet and coordinated through the API server + etcd.
3. The K8s scheduler extends OS scheduling to **cluster scale**: assign Pods to nodes instead of threads to CPUs.
4. **Filter → Score → Bind** separates hard feasibility from soft policy — the same separation as admission control vs. optimization in OS schedulers.
5. Taints, tolerations, and affinity give fine-grained control over placement — analogous to CPU pinning and NUMA affinity in a kernel.
6. Fairness, packing, and fragmentation make cluster scheduling harder than CPU scheduling — multiple resources, stale views, and node failures add complexity.
7. All scheduling decisions are **persisted through etcd** (Week 7), making them durable, consistent, and recoverable.

Literature — Foundational Papers

Topic	Reference	Why it is useful for this week
Production cluster management	Verma et al., **"Large-scale cluster management at Google with Borg"** (2015)	The best primary source for why large clusters need scheduler-driven placement, priorities, packing, and admission control
Design lineage	Burns et al., **"Borg, Omega, and Kubernetes"** (2016)	The most teaching-friendly bridge from Borg/Omega ideas to Kubernetes
Cluster-scheduler overview	Schwarzkopf, **"Cluster Scheduling for Data Centers"** (2017)	A compact overview that helps students connect OS scheduling ideas to cluster-scale scheduling
Fragmentation and packing	Grandl et al., **"Multi-resource packing for cluster schedulers"** (2014)	Useful background for the packing vs. spreading and fragmentation discussion
Kubernetes scheduling survey	Senjab et al., **"A survey of Kubernetes scheduling algorithms"** (2023)	Good follow-up reading for students who want modern scheduler variants and extensions

---

Literature — Practical References

For implementation details and lab follow-up, the official Kubernetes docs are also worth reading:

- **Kubernetes Scheduler**
  https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/
- **Scheduling Framework**
  https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/
- **Assigning Pods to Nodes**
  https://kubernetes.io/docs/concepts/scheduling-eviction/assign-pod-node/
- **Taints and Tolerations**
  https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/
- **Pod Priority and Preemption**
  https://kubernetes.io/docs/concepts/scheduling-eviction/pod-priority-preemption/

End

> **Docker:** how do I run one container on one machine?
>
> **Kubernetes:** who decides where thousands of containers run across hundreds of machines — and how does that decision become shared truth?